Defesa de Viabilidade — Câmara Climatizada (Plataforma SAGA SENAI)

Título do Projeto

Modernização e Supervisão Inteligente para Cabine Climatizada de Ensaios Térmicos

---

1. Qual é a demanda da empresa?

A empresa realiza testes térmicos em dispositivos eletrônicos. Ela coloca até **50 aparelhos** dentro de uma cabine climatizada, liga tudo e espera horas enquanto a câmara esquenta e esfria em ciclos programados.

O problema é que, durante esse tempo todo, **ninguém sabe se um dos aparelhos parou de funcionar**. Se um dispositivo queima no meio do teste, o operador só descobre no final — às vezes depois de 8 horas. Aí não dá para saber **qual** aparelho falhou, **quando** falhou, e o teste inteiro pode ser descartado.

A empresa quer uma solução que:

- Controle a temperatura automaticamente (sem depender de alguém ajustando manualmente).
- **Monitore cada aparelho individualmente** e avise imediatamente se algum parar de funcionar.
- Registre tudo com data e hora, para ter um histórico confiável.
- Permita acompanhar o teste à distância, pelo computador ou celular.

---

2. Por que escolhemos essa demanda?

Escolhemos essa demanda porque ela encaixa perfeitamente no que aprendemos no curso de **Eletrotécnica**:

- **Comandos elétricos:** O projeto usa relés, botão de emergência e selo elétrico — exatamente o que estudamos em aula. A segurança é feita em hardware: se o programa travar, a máquina desliga sozinha.
- **Instalações elétricas:** Precisamos dimensionar fios, disjuntores, fusíveis e fontes de alimentação. O painel tem 3 trilhos DIN com separação entre potência e sinal, como num quadro industrial de verdade.
- **Instrumentação:** Usamos sensores de temperatura e de corrente elétrica. O sensor de corrente é o que permite saber se cada aparelho está funcionando ou não.
- **Automação:** O controle de temperatura funciona em **malha fechada com PID** — o sistema mede, compara com o valor desejado e ajusta sozinho (explicamos o que é PID na seção 10).
- **Redes industriais:** Os sensores de corrente se comunicam por **RS-485**, que é o mesmo barramento usado nas fábricas (explicamos como funciona na seção 11). Isso prova que o projeto não é só escolar: ele usa a mesma tecnologia da indústria.

Em resumo: não é um projeto só de programação. É um projeto de **eletrotécnica com programação**, que é exatamente o perfil do nosso curso.

---

3. Como funciona a nossa solução?

A ideia é simples: colocamos um **sensor de corrente** em cada posição de teste. Se o aparelho está funcionando, passa corrente. Se parou, a corrente cai para zero. Quando isso acontece, o sistema avisa na hora.

Na maquete (o que vamos construir):

- Em vez de uma cabine industrial gigante, usamos uma **caixinha de acrílico isolada** com pastilhas Peltier para esfriar (aquelas "pedrinhas" que transferem calor de um lado pro outro) e um **aquecedor PTC** para esquentar (uma peça cerâmica que gera calor quando ligada na corrente). Ela é pequena, mas funciona igualzinho: esfria, esquenta e controla a temperatura.
- No lugar dos 50 aparelhos, usamos **2 lâmpadas automotivas** como carga de teste. Se a lâmpada queima ou se puxamos o fusível, o sensor detecta que a corrente sumiu e dispara o alarme.
- Usamos **módulos de sensor de corrente com furo de passagem** (como a linha WCS2702). Assim, não precisamos cortar o fio de alimentação para medir a corrente contínua (DC) das lâmpadas — o fio só passa por dentro do furo. *(Nota: se os aparelhos testados pela empresa funcionarem na tomada comum, em Corrente Alternada - AC, o projeto continua idêntico. Nós só trocamos esse módulo por um TC miniatura de corrente alternada).*
- O painel de comando é de verdade: trilho DIN, canaletas, relés, botão de emergência, sinaleiros — tudo como num painel industrial.

O que acontece passo a passo:

1. O operador configura o teste na **IHM** (a tela de interface do painel, com toque na tela): escolhe a temperatura desejada e o tempo do ensaio.

2. O Arduino controla a temperatura automaticamente em malha fechada PID: mede a temperatura, compara com o que o operador pediu, e liga a Peltier (se precisa esfriar) ou o PTC (se precisa esquentar).
3. Os sensores de corrente monitoram as lâmpadas o tempo todo. O sinal vai por RS-485 até o ESP32, que fica perguntando "como estão as correntes?" a cada segundo.
4. Se uma lâmpada apaga, o sistema identifica **qual** posição falhou e **quando**, grava no cartão de memória e manda um aviso pelo Wi-Fi.
5. O operador pode acompanhar tudo pelo computador (dashboard), sem precisar ficar na frente do painel.

---

4. É viável? Dá pra fazer?

Sim, e por 3 motivos:

1. **Custo baixo:** Todos os componentes são vendidos no Brasil e custam pouco. O protótipo inteiro (maquete + painel + câmara + sensores) fica abaixo de **R$ 2.000,00**.

2. **Componentes prontos:** Não estamos inventando nada do zero. Usamos Arduino, ESP32, sensores e módulos que já existem prontos no mercado. É só montar e programar.

3. **Construção por etapas:** O projeto é dividido em 5 camadas (Fundamentos → Maquete → Painel → Elétrica → Integração). Se der problema numa etapa, não perde o que já fez. Dá para testar cada pedaço separado antes de juntar tudo.

---

5. E na fábrica real? Quanto custa implementar de verdade?

A fábrica **já tem o Arduino** controlando a câmara. O que falta é o sistema de monitoramento dos 50 dispositivos. Não dá para ligar 50 sensores direto no ESP32 — seriam fios demais e o ESP32 não tem 50 entradas. A solução é usar **módulos concentradores RS-485**.

Como funciona na indústria:

- Cada grupo de **8 sensores** é ligado a um **módulo ADC de 8 canais** (um "tradutor" que lê os 8 sensores e empacota tudo num único dado digital).
- Esses módulos ficam espalhados no quadro de disjuntores, **perto dos sensores** (fios curtos, sem ruído).
- Todos os módulos são ligados entre si por apenas **2 fios** (o barramento RS-485), um atrás do outro, como uma corrente.
- Na ponta dessa corrente, o ESP32 fica perguntando para cada módulo: "módulo 1, como estão seus 8 sensores?", "módulo 2, como estão os seus?" — e assim por diante.
- Com **7 módulos de 8 canais** dá para monitorar os 50 dispositivos (7 × 8 = 56 posições).

Estimativa de custo para a fábrica (50 posições):

Item	Quantidade	Custo unitário	Subtotal
Sensor de corrente tipo "alicate" (TC Split-Core, não precisa cortar o fio)	50 unidades	R$ 30 a R$ 50	R$ 1.500 a R$ 2.500
Módulo ADC 8 canais Modbus RS-485 (concentrador)	7 unidades	R$ 180 a R$ 300	R$ 1.260 a R$ 2.100
ESP32 industrial com RS-485 e Ethernet (tipo DNLCB30 ou ESP32-P4)	1 unidade	R$ 150 a R$ 300	R$ 150 a R$ 300
Cabo par trançado blindado RS-485 (~50 a 100 m)	1 rolo	R$ 150 a R$ 300	R$ 150 a R$ 300
Resistores de terminação 120 Ω, bornes, conectores	—	—	R$ 100 a R$ 200
Software do dashboard (Grafana / Node-RED)	—	**Gratuito**	R$ 0
TOTAL ESTIMADO			**R$ 3.300 a R$ 5.400**

> Esse valor é para monitorar **todos os 50 dispositivos de uma vez**. Comparado com o custo de descartar lotes inteiros de ensaio porque ninguém viu a falha, o investimento se paga rápido.

Opção de evolução: Arduino UNO R4 WiFi

Na maquete, usamos 3 peças separadas que precisam conversar entre si por fios: o **Arduino Mega** (que controla a temperatura), o **ESP32** (que cuida do Wi-Fi e do Modbus) e o **módulo RTC DS3231** (que guarda a data e hora). São 3 placas, 3 alimentações, fios ligando uma na outra — mais pontos onde algo pode dar errado.

O **Arduino UNO R4 WiFi** resolve isso juntando tudo numa placa só:

- Tem um **processador Renesas** forte o bastante para rodar o PID (o controle de temperatura).
- Tem um **ESP32-S3 embutido** que já cuida do Wi-Fi e do Bluetooth — sem precisar de módulo separado.
- Tem **RTC interno** (relógio de tempo real) — sem precisar do módulo DS3231 externo.
- Tem **USB-C** moderno em vez do USB-B antigo do Mega.
- Se programa na mesma **IDE do Arduino** com a mesma linguagem — o código muda pouco.

Ou seja: **2 componentes viram 1** (Arduino + ESP32 viram uma placa só), e o RTC sai de graça. Menos fios, menos pontos de falha, menos espaço no painel.

Na fábrica, onde já existe Arduino, trocar o controlador atual por um UNO R4 WiFi significaria ganhar Wi-Fi, RTC e capacidade de ser o mestre Modbus RS-485 **sem adicionar nenhum módulo extra**. O custo de um UNO R4 WiFi fica em torno de **R$ 200 a R$ 280** — que é menos do que comprar um ESP32 industrial separado mais o RTC mais o conversor de nível lógico, e ainda sobra a economia de não precisar dos fios entre as placas.

Abordagem	Componentes	Custo estimado
Atual (maquete)	Arduino Mega + ESP32 + RTC + conversor de nível	~R$ 350 (4 peças + fios entre elas)
Evolução	**Arduino UNO R4 WiFi** (tudo em 1 placa)	~R$ 200 a R$ 280 (1 peça)

---

6. Quais são os riscos?

E se...	O que fazemos
Um componente atrasa para chegar?	O projeto é modular: dá para testar as outras partes enquanto espera.
O sensor erra a leitura?	Calibramos em bancada e colocamos filtros no software para ignorar leituras absurdas.
O dissipador esquenta demais?	Os coolers têm sensores de rotação. Se parar de girar, o sistema desliga a Peltier na hora.
O Wi-Fi cai?	O controle continua funcionando sozinho. O log fica gravado no cartão de memória. Quando o Wi-Fi volta, os dados são reenviados.

---

7. Qual é o diferencial?

Câmaras climatizadas comuns controlam só a temperatura. A nossa proposta vai além: **monitora cada aparelho individualmente**. Se um dos 50 dispositivos morre no meio do teste, o operador sabe na hora qual foi e em que momento — em vez de descobrir 8 horas depois, quando o teste já acabou.

Isso é **rastreabilidade**, e é o que transforma um ensaio genérico em um processo controlado e confiável.

---

8. Divisão da apresentação (5 integrantes, ~2 min cada)

Quem	Fala sobre	Frase-chave
Integrante 1	O problema da empresa e por que ele importa	"O operador não sabe qual aparelho falhou nem quando. Nosso projeto resolve isso."
Integrante 2	A parte elétrica: painel, emergência, proteções	"A segurança é em hardware. Se o programa travar, a máquina desliga sozinha."
Integrante 3	O controle de temperatura (PID) e os sensores	"O PID mede, compara e ajusta sozinho. E a corrente de cada aparelho é vigiada o tempo todo pelo RS-485."
Integrante 4	O software, a IHM, o Wi-Fi e os dados	"Tudo é registrado com data e hora. Se a rede cair, o sistema continua funcionando sozinho."
Integrante 5	Custos, viabilidade e escalabilidade industrial	"O protótipo custa menos de R$ 2.000. Na fábrica, com R$ 3.300 a R$ 5.400 monitora os 50 dispositivos."

---

9. O que todo mundo do grupo precisa saber responder

1. **Qual é o problema?** O teste térmico roda por horas e ninguém sabe se um aparelho parou de funcionar no meio.
2. **Qual é a solução?** Um painel com sensores de corrente que monitora cada posição individualmente e avisa na hora se algo falhar.
3. **É seguro?** Sim. O botão de emergência e as proteções funcionam em hardware, sem depender do programa.
4. **Quanto custa?** Menos de R$ 2.000 o protótipo. Na fábrica real, de R$ 3.300 a R$ 5.400 para monitorar os 50 dispositivos com módulos RS-485.
5. **Escala pra fábrica?** Sim. Usa 7 módulos de 8 canais ligados em RS-485, um ESP32 industrial na ponta e o dashboard é gratuito.
6. **O que é PID?** É o controle automático de temperatura: mede, compara com o desejado e ajusta. Igual o ar-condicionado de casa, só que feito por nós (ver seção 10).
7. **O que é RS-485?** É um barramento industrial de 2 fios que permite ligar vários módulos em série e ler todos com um único controlador (ver seção 11).
8. **Qual é o diferencial?** Saber **qual** aparelho falhou e **quando**, com registro automático — e não só controlar a temperatura.

---

10. O que é malha fechada e PID? (para a banca perguntar)

Imagine o ar-condicionado da sua casa. Você coloca em 22 °C e ele faz o resto sozinho: esfria quando está quente, desliga quando chega na temperatura, liga de novo se esquentar. Ele **mede** a temperatura, **compara** com o que você pediu e **age** para corrigir a diferença. Isso é uma **malha fechada** — o sistema se autoajusta.

Agora, o **PID** é a "receita" que o sistema usa para decidir **quanto** esfriar ou esquentar. As três letras significam:

- **P (Proporcional):** "Quanto mais longe eu estou da temperatura certa, mais forte eu ligo." Se falta muito para chegar em 22 °C, ele liga no máximo. Se falta pouco, liga devagar.
- **I (Integral):** "Se eu estou pertinho mas nunca chego exatamente, vou aumentando um pouquinho até chegar." O I corrige erros pequenos que o P sozinho não consegue zerar.
- **D (Derivativo):** "Se a temperatura está caindo rápido e vai passar do ponto, vou freando antes." O D evita que o sistema passe do valor desejado e fique oscilando.

Na prática do nosso projeto: O Arduino mede a temperatura da câmara a cada 1 segundo. Se está acima do desejado, liga a Peltier (frio). Se está abaixo, liga o PTC (calor). O PID calcula exatamente quanto de potência enviar (de 0% a 100%) para chegar na temperatura certa sem ficar ligando e desligando feito louco.

> **Por que isso importa pro curso?** Porque malha fechada e PID são a base de qualquer automação industrial: controle de nível de tanques, velocidade de motores, pressão de caldeiras. Aprender na câmara térmica serve pra qualquer processo.

---

11. O que é RS-485 e como funciona? (para a banca perguntar)

Imagine que você tem 50 sensores espalhados num quadro elétrico e quer ler todos eles com um único ESP32. Não dá para ligar 50 fios analógicos no ESP32 (ele não tem 50 entradas), e mesmo se tivesse, os fios longos pegariam interferência dos motores e dariam leituras erradas.

A solução é o **RS-485**: um barramento industrial que funciona com **apenas 2 fios** (chamados A e B).

Como funciona, passo a passo:

1. **Os sensores são agrupados:** A cada 8 sensores, tem um "módulo concentrador" (módulo ADC). Ele lê os 8 sensores que estão perto dele e guarda os valores na memória.
2. **Os módulos são ligados em corrente:** O fio A do primeiro módulo liga no A do segundo, que liga no A do terceiro... e assim por diante. São só 2 fios percorrendo todos os módulos, um atrás do outro (isso se chama **daisy-chain**, ou "corrente em margarida").
3. **O ESP32 pergunta um por um:** O ESP32 é o "mestre" da rede. Ele manda uma mensagem: "Módulo 1, me diz seus 8 valores". O módulo 1 responde. Depois pergunta pro módulo 2, pro 3, e assim por diante. Esse esquema de perguntar e responder se chama **Modbus RTU** — é um protocolo padrão usado no mundo inteiro em fábricas.
4. **Por que funciona bem em ambiente industrial?** Porque o RS-485 não manda tensão absoluta (tipo 0 V ou 5 V, que qualquer motor por perto bagunça). Ele manda a **diferença** de tensão entre os 2 fios. Se o ruído sobe nos dois fios ao mesmo tempo, a diferença fica igual — o ruído se cancela sozinho. Por isso o RS-485 funciona bem a **distâncias de até 1.200 metros**, mesmo perto de motores, compressores e inversores de frequência.
5. **Por que não usar sensores 4-20 mA para longas distâncias?**
   A indústria ama sensores 4-20 mA porque eles também andam longas distâncias sem perder o sinal. Mas se fôssemos usar transmissores 4-20 mA para 50 aparelhos, esbarraríamos em três problemas graves:

- **Custo e Cabos:** Seriam 50 transmissores muito caros e 50 cabos cruzando a fábrica até o painel.
- **Portas limitadas:** O Arduino Mega só tem 16 entradas analógicas. O ESP32 tem menos ainda. Não teríamos onde ligar 50 fios analógicos.
- **Leitura incompatível:** Microcontroladores só leem *Tensão* (0 a 5 Volts). Para ler Corrente (4-20 mA), precisaríamos instalar 50 resistores de precisão ou 50 plaquinhas conversoras no painel. No fim das contas, para o Arduino ler os 50 sensores 4-20 mA, teríamos que usar módulos concentradores RS-485 de qualquer jeito!
  Por isso nossa solução corta caminho: usamos sensores simples e baratos perto das lâmpadas, ligamos eles direto nos módulos RS-485 lá na câmara, e puxamos **um único cabo de 2 fios** cruzando a fábrica. Economizamos muito dinheiro e resolvemos a falta de portas do Arduino.

No nosso protótipo (maquete):

- Temos 2 sensores de corrente ligados num módulo ADC de 2 canais.
- O módulo ADC se comunica com o ESP32 pelo RS-485, usando Modbus.
- É a mesma coisa que a fábrica faria com 50 sensores e 7 módulos — só que em escala menor.

> **Por que isso importa pro curso?** Porque RS-485 e Modbus são a base de comunicação de qualquer automação industrial: CLPs, inversores de frequência, medidores de energia, IHMs. Aprender a usar isso na maquete é aprender a linguagem que as máquinas da fábrica falam.

---

12. Perguntas "Pegadinha" que a banca pode fazer (e como responder)

Se a banca quiser testar se vocês realmente entendem de eletrotécnica e não só de programação, eles farão perguntas difíceis. Aqui estão as principais e como responder:

Pergunta 1: "Por que usar 24 Volts para alimentar as Peltier e o Aquecedor, e não 110V ou 220V direto da tomada?"
Como responder: "Por segurança e norma (NR-10). Como é uma maquete didática que alunos vão operar, optamos por trabalhar em Extra-Baixa Tensão (SELV - 24V DC). Isso elimina o risco de choque elétrico letal durante a montagem e os testes. Na fábrica real, o comando continua em 24V, mas a potência seria em 220V/380V para os compressores. A nossa maquete simula a lógica de comando perfeitamente, mas com segurança total."

Pergunta 2: "O que acontece se o Arduino travar enquanto o aquecedor estiver ligado no máximo? Vai pegar fogo?"
Como responder: "Não. Nós pensamos no cenário de falha de software. Primeiro, o Arduino tem um *Watchdog* (um cão de guarda interno): se o programa travar, ele reinicia o chip em 2 segundos. Segundo, nós instalamos resistores de *pull-down* nos gatilhos de potência. Se o Arduino desligar ou travar, a energia vai para zero volts, e as chaves de potência desligam o aquecedor fisicamente. E, em último caso, o botão de emergência corta a energia direto no fio (em hardware), ignorando o Arduino."

Pergunta 3: "Se isso é para a indústria, por que usar Arduino e não um CLP (Controlador Lógico Programável) de mercado?"
Como responder: "Para fins didáticos, o Arduino e o ESP32 nos forçam a aprender a lógica de baixo nível (como calcular um PID matematicamente e como montar a rede Modbus do zero), conhecimentos que o CLP esconde do operador. Além disso, nós documentamos no projeto que a lógica migra perfeitamente para CLPs baseados em Arduino que já são certificados para a indústria, como o *Arduino Opta* ou o *Portenta Machine Control*, aproveitando o mesmo código."

---

13. Encerramento sugerido

"Concluímos que o projeto é viável porque usa componentes baratos e comerciais, a montagem é por etapas e a segurança não depende de software. Ele aplica na prática o que aprendemos em comandos elétricos, instalações, instrumentação e automação. E o mais importante: resolve um problema real da empresa — saber na hora quando um dispositivo falha, em vez de descobrir horas depois. Na fábrica, com um investimento de R$ 3.300 a R$ 5.400, o sistema monitora os 50 dispositivos usando a mesma arquitetura RS-485 que provamos funcionar na maquete."

Defesa de Viabilidade — Silos de Gipsita (Plataforma SAGA SENAI)

> **Demanda SAGA:** #13063 — Silos inteligentes para controle de volume de material (Gipsita)

Título do Projeto

Sistema de Telemetria Inteligente para Medição de Nível em Silos de Gipsita

---

1. Qual é a demanda da empresa?

A empresa fabrica gesso a partir de um mineral chamado **gipsita**. Essa gipsita fica armazenada em silos grandes (tipo "caixas d'água" gigantes de metal). O problema é que, para saber quanto material ainda tem dentro do silo, **alguém precisa subir lá em cima e olhar**.

Isso causa 3 problemas sérios:

- **Risco de acidente:** Subir num silo de vários metros de altura, num ambiente cheio de pó de gesso, é perigoso. Pode cair, pode respirar poeira demais.
- **Falta de controle:** Sem saber em tempo real quanto material tem, a fábrica pode parar porque acabou a gipsita no silo — ou pode transbordar porque encheu demais.
- **Sensores comuns não funcionam:** A empresa já tentou usar sensores de nível, mas a poeira dentro do silo é tão densa que os sensores normais (que usam som ou laser) perdem o sinal e dão leituras erradas.

A empresa quer uma solução que:

- Meça o nível de gipsita **sem precisar de alguém subir no silo**.
- Funcione **mesmo com muita poeira** dentro do silo.
- Mostre os dados **em tempo real** num painel na sala de controle.

---

2. Por que escolhemos essa demanda?

Escolhemos porque ela é um problema real de uma empresa real, e a solução usa exatamente o que aprendemos no curso de **Eletrotécnica**:

- **Instalações elétricas:** Precisamos montar um painel elétrico protegido contra poeira e água (IP65), com disjuntor, proteção contra surtos e fonte de alimentação. Tudo dimensionado corretamente.
- **Instrumentação industrial:** Usamos um sensor de nível por radar e um sinal de corrente de **4 a 20 mA** — que é o padrão da indústria para transmitir medições por fios longos sem perder precisão.
- **Automação e IoT:** O microcontrolador ESP32 recebe o sinal do sensor, calcula o volume do silo e envia os dados pela rede Wi-Fi para um painel que os gestores podem ver no computador.
- **Segurança no trabalho:** O projeto elimina a necessidade de subir no silo, reduzindo o risco de acidentes em altura (NR-35).

Em resumo: é um projeto de **eletrotécnica aplicada à instrumentação industrial**, com um bônus de segurança no trabalho.

---

3. Por que radar e não outro sensor?

Essa é a parte mais importante de explicar. Dentro de um silo de gipsita, durante o enchimento, o ar fica cheio de uma nuvem densa de pó de gesso:

- **Sensor de ultrassom** (que usa som): O som bate na nuvem de poeira e volta antes de chegar no material. Resultado: leitura errada.
- **Sensor a laser** (que usa luz): A poeira suja a lente em poucas horas e bloqueia o feixe. Resultado: precisa limpar todo dia.
- **Sensor de radar** (que usa ondas de rádio): As ondas de rádio de alta frequência **atravessam a poeira** como se ela não existisse. A antena do sensor é revestida de Teflon (PTFE), então o pó não gruda. Resultado: funciona sem manutenção.

Por isso, a indústria de silos e mineração no mundo inteiro usa radares. É a tecnologia certa para o problema certo.

---

4. Como funciona a nossa solução?

A ideia é bem simples:

1. **O sensor fica no topo do silo**, apontando para baixo. Ele dispara uma onda de rádio que bate na gipsita e volta. Pela velocidade da onda, ele calcula a **distância** até a superfície do material.
2. Essa distância é convertida num sinal elétrico de **4 a 20 mA** (um padrão industrial que não perde precisão mesmo com cabos longos).
3. O fio chega num **painelzinho elétrico** na base do silo, onde um módulo conversor transforma esse sinal em número digital.
4. O **ESP32** (microcontrolador com Wi-Fi) pega esse número, faz a conta de volume do silo (considerando o formato cilíndrico/cônico) e transforma em **porcentagem de lotação**.
5. Esse dado é enviado pela rede Wi-Fi para um **painel na tela do computador** na sala de controle, onde o gestor vê em tempo real: "Silo 1: 73%", "Silo 2: 28%", etc.
6. Se o nível ficar muito baixo (risco de parar a produção) ou muito alto (risco de transbordar), o sistema **avisa automaticamente**.

Na maquete (o que vamos construir):

- Um tubo de PVC simulando o silo, com gipsita moída dentro.
- Um sensor radar pequeno no topo (versão didática, que custa uns R$ 100).
- O ESP32 calculando o volume e mostrando num dashboard no computador.
- A mesma lógica que seria usada na fábrica real, só que em escala reduzida.

---

5. É viável? Dá pra fazer?

Sim, e é mais simples do que parece:

1. **Custo baixo:** O protótipo de maquete inteiro custa cerca de **R$ 330,00** (sensor + ESP32 + painel + materiais). Se o grupo conseguir mais orçamento, dá para usar um radar industrial de verdade por uns R$ 1.300,00.

2. **Componentes prontos:** Sensores radar para medir distância já existem prontos no mercado. O ESP32 é um módulo que já vem com Wi-Fi. O software para o dashboard (Grafana ou Node-RED) é gratuito.

3. **Construção rápida:** O projeto tem 4 fases:

    - Fase 1: Pesquisar e comprar os componentes.
    - Fase 2: Montar o painel e testar o sensor na bancada.
    - Fase 3: Programar o ESP32 e o dashboard.
    - Fase 4: Testar na maquete do silo e validar.

E na fábrica real?
Na fábrica, em vez do sensor didático de R$ 100, usaria um radar industrial de 80 GHz (R$ 3.000 a R$ 8.000 por silo), homologado para ambientes com poeira. Mas a lógica do ESP32, o protocolo MQTT e o dashboard seriam **exatamente os mesmos**. Só troca o sensor.

---

6. Quais são os riscos?

E se...	O que fazemos
O Wi-Fi da fábrica não chegar até o silo?	Usamos cabo de rede em conduíte metálico ou rádio LoRa de longo alcance.
O sol esquentar demais o painel?	O painel fica num local coberto ou com proteção anti-UV.
O sensor didático não tiver a mesma precisão do industrial?	Documentamos a limitação e apresentamos a especificação do sensor real que seria usado na fábrica.
A conta de volume der errada?	Calibramos com medição manual de referência e colocamos margem de tolerância no software.

---

7. Qual é o diferencial?

A maioria das fábricas de gesso ainda mede o nível dos silos **na mão**, com alguém subindo lá em cima. As que tentaram automatizar falharam porque usaram sensores errados (ultrassom e laser), que não funcionam com poeira.

Nosso projeto propõe a tecnologia certa (radar) com o custo mais baixo possível (ESP32 + Wi-Fi + MQTT), entregando:

- **Segurança:** ninguém mais precisa subir no silo.
- **Controle:** o gestor vê em tempo real quanto material tem.
- **Previsibilidade:** alarmes automáticos avisam antes de faltar ou transbordar.

---

8. Divisão da apresentação (5 integrantes, ~2 min cada)

Quem	Fala sobre	Frase-chave
Integrante 1	O problema da empresa e por que importa	"O operador precisa subir no silo para ver o nível. É perigoso e impreciso. Nosso projeto resolve isso."
Integrante 2	Por que radar e não ultrassom ou laser	"A poeira engana o ultrassom e suja o laser. Só o radar atravessa a nuvem de pó sem problema."
Integrante 3	O painel elétrico e a infraestrutura	"Montamos um painel IP65 protegido contra poeira e água, com proteção contra surtos elétricos."
Integrante 4	O software, o cálculo de volume e o dashboard	"O ESP32 transforma a distância em volume e manda para o computador da sala de controle em tempo real."
Integrante 5	Custos, viabilidade e escalabilidade	"O protótipo custa R$ 330. Na fábrica, troca só o sensor. A lógica e o software são os mesmos."

---

9. O que todo mundo do grupo precisa saber responder

1. **Qual é o problema?** A fábrica não sabe quanto material tem nos silos sem mandar alguém subir lá.
2. **Por que sensores normais não servem?** A poeira de gipsita engana o som e suja o laser. Só radar funciona.
3. **Como funciona?** O radar mede a distância, o ESP32 calcula o volume e envia por Wi-Fi para um painel no computador.
4. **É seguro?** Sim. Painel IP65, proteção contra surtos, e ninguém mais precisa subir no silo.
5. **Quanto custa?** R$ 330 a maquete. R$ 1.300 com radar industrial. R$ 3.000–8.000 por silo na fábrica real.
6. **Escala pra fábrica?** Sim. Troca só o sensor pelo industrial. O ESP32, o software e o dashboard ficam iguais.
7. **Qual é o diferencial?** Medir o nível sem contato, sem subir no silo, imune à poeira, com dados em tempo real.

---

10. Perguntas "Pegadinha" que a banca pode fazer (e como responder)

Como esse é um projeto industrial voltado para o setor de mineração/gesso, a banca fará perguntas para testar se a solução para em pé na vida real.

Pergunta 1: "O silo é uma estrutura de metal gigante que funciona como Gaiola de Faraday, além do ambiente ser cheio de poeira. Como o Wi-Fi do ESP32 vai chegar na sala de controle?"
Como responder: "Na maquete nós usamos Wi-Fi, mas na fábrica nós documentamos opções mais robustas. O painel ficaria na base do silo (do lado de fora), ou usaríamos uma antena Wi-Fi externa apontada para um roteador industrial próximo. Se o Wi-Fi não for viável, a arquitetura permite trocar o módulo Wi-Fi por um Rádio LoRa (915 MHz, que atravessa obstáculos muito melhor) ou simplesmente passar um cabo de rede blindado até o switch mais próximo."

Pergunta 2: "Um radar industrial custa de R$ 3.000 a R$ 8.000. Vocês acham que a empresa vai pagar isso só para evitar que o operador suba uma escada?"
Como responder: "Com certeza. Primeiro, porque uma queda de altura (NR-35) pode ser fatal, e isso tem um custo humano imensurável, além de processos milionários. Segundo, o silo ficar vazio significa que os fornos da fábrica param. Uma hora de fábrica parada custa muito mais que 8 mil reais. O sensor se paga na primeira vez que evita a interrupção da produção."

Pergunta 3: "O sensor industrial envia o sinal em 4 a 20 mA. Mas o ESP32 só lê tensão de 0 a 3.3V. Na indústria isso liga direto no CLP, mas como vocês vão ligar no ESP32 sem queimar a placa?"
Como responder: "Temos duas opções. A mais profissional é usar um **Módulo Conversor 4-20mA para Tensão** (como o HW-685). Ele recebe os fios industriais 24V/20mA de um lado e entrega 3.3V seguros para o ESP do outro. Mas, se precisarmos fazer na raça, usamos a **Lei de Ohm** ($V = R \times I$): colocamos um resistor de precisão (shunt) de 165 ohms em paralelo com a entrada do ESP32 ligada ao GND. Quando o sensor manda a corrente máxima de 20 mA ($0.02 A$), a tensão gerada sobre o resistor será de exatamente $3.3V$ (165×0.02), que é o limite seguro do pino do ESP32."

---

11. Encerramento sugerido

"Concluímos que o projeto é viável porque usa uma tecnologia comprovada na indústria — o radar — combinada com IoT de baixo custo. Ele resolve um problema real de segurança e logística da empresa, aplica os conhecimentos de instalações elétricas, instrumentação e automação do nosso curso, e pode ser validado com uma maquete de R$ 330 que funciona com a mesma lógica da planta real."

04. Escalabilidade: Maquete vs. Indústria

> "Como provar que um projeto didático de bancada realmente se aplica a um chão de fábrica?"

Para fins didáticos e de custos, nossa **Maquete** utiliza componentes compactos (como Arduino Mega, ESP32 e sensores ACS712). Porém, a arquitetura foi desenhada desde o início para ser idêntica à de uma **Planta Industrial**, mudando apenas os hardwares (instrumentos) utilizados.

A fábrica já possui um ecossistema padronizado, muitas vezes utilizando o próprio padrão Arduino em hardwares de nível industrial (PLCs e Controladores Pro robustos).

Abaixo, documentamos o comparativo exato entre o que está montado na **Maquete (Didático)** e o que será comprado/instalado na **Indústria (Planta Real)**.

---

1. O Controlador Principal e IoT

Na maquete, nós dividimos as tarefas em três placas isoladas que precisam conversar entre si: o **Arduino Mega 2560** (Controle PID e lógica), o **ESP32** (Rede Wi-Fi, Modbus, MQTT e IHM) e um módulo **RTC DS3231** avulso para segurar a hora.

Na **Indústria**, manter módulos separados gera pontos de falha (fios soltos, ruído em I2C, falha de solda). A solução profissional unifica tudo numa única placa robusta, já padronizada no chão de fábrica:

Função	Na Maquete (Didático)	Na Indústria (Planta Real)
Cérebro e I/O	Arduino Mega 2560	**Arduino Portenta Machine Control** (ou **Arduino GIGA R1 WiFi** / **Arduino Opta WiFi**)
Conectividade (IoT)	Módulo ESP32 + MAX485	**Nativo na placa** (Ethernet / Wi-Fi / RS-485 embutidos)
Relógio (RTC)	Módulo DS3231 externo	**Nativo na placa** (RTC de hardware interno com bateria backup tipo botão CR2032)
Armazenamento (Log)	Módulo SD Card SPI	**Nativo na placa** (Entrada microSD e/ou Flash nativa de alta capacidade)
Interface USB	USB Tipo-B (Serial)	**USB Tipo-C** / **USB Host** nativo
Confiabilidade	Fios Jumper Dupont	**Bornes industriais de mola ou parafuso direto na placa**

> 📌 **A Evolução Industrial:** Placas da linha **Arduino Pro** (como o *Portenta Machine Control* ou a linha *Opta*) reúnem todo o poder do Arduino, combinam co-processadores de rede e já entregam portas RS-485 e Ethernet de fábrica num trilho DIN, sem jumpers. Se o cliente exige uma placa estilo devboard de altíssimo poder, o **ESP32-P4** (com expansões Ethernet PHY) ou o **Arduino GIGA R1 WiFi** (que tem o tamanho do Mega, mas tem RTC, USB-C e co-processador Wi-Fi) cumprem essa função magistralmente numa única peça.
>
> 🔄 **Evolução intermediária: Arduino UNO R4 WiFi.** Para quem já usa Arduino e quer dar o primeiro passo sem trocar toda a plataforma, o **UNO R4 WiFi** reúne processador Renesas (Arm Cortex-M4), co-processador **ESP32-S3** embutido (Wi-Fi + Bluetooth), **RTC interno** e USB-C — tudo numa única placa, programável na mesma IDE Arduino. Ele substitui o Arduino Mega + ESP32 + RTC DS3231 por uma peça só, mantendo compatibilidade com o código existente e reduzindo pontos de falha.

---

2. Monitoramento das 50 Posições de Ensaio

O desafio principal é monitorar 50 disjuntores de cargas sem poluir o quadro com 100 fios analógicos correndo por dezenas de metros até o controlador. A arquitetura de rede **Modbus RTU via RS-485** é a mesma em ambos os mundos. O que muda é como medimos.

Item	Na Maquete (2 posições)	Na Indústria (50 posições)
A Carga do Teste	Lâmpadas Automotivas W5W Pingo (24V 5W) gerando calor e corrente.	Equipamentos Reais sob ensaio nas esteiras (cargas AC pesadas).
Proteção de Ramal	Porta-Fusíveis Seccionáveis DIN (Fusível de Vidro 500mA).	Minidisjuntores Curva C de força.
O Sensor de Corrente	**Módulo Efeito Hall Não-Invasivo** com furo de passagem (ex: WCS2702). Permite medir DC sem cortar o fio. *(Se a carga de teste na maquete for AC, basta usar um TC miniatura tipo ZMCT103C).*	**Transformador de Corrente (TC) Split-Core** ou **Sensor SZC23**: Não invasivo (abotoado por fora do cabo). Mede AC pesada.
Digitalização do Sinal	**Módulo ADC 2 Canais** (Conversor A/D 0-5V para Modbus RTU).	**Módulos ADC 8AI (ou 16AI) Industriais** de trilho DIN. Cada módulo concentra 8 TCs locais e despacha tudo via rede RS-485.

Por que a arquitetura Modbus é a prova da maturidade do projeto?

Num chão de fábrica, tentar levar 50 fios analógicos de tensão (0 a 5V) por 20 metros resultaria em leituras completamente malucas devido à interferência magnética de motores e compressores.

E por que não usar o padrão industrial 4-20 mA?
O sinal 4-20 mA é imune a ruídos e viaja longas distâncias, mas se a empresa fosse utilizar transmissores 4-20 mA para monitorar 50 aparelhos, o custo seria altíssimo. Seriam 50 sensores caros e a necessidade de puxar 50 cabos independentes cruzando todo o galpão até chegar no painel do controlador principal (CLP/Arduino).

A nossa arquitetura resolve isso com o **RS-485**. Nós mantemos os sensores puros e baratos (TCs normais) instalados perto das cargas, e digitalizamos o sinal ali mesmo através de Módulos ADC **locais** de trilho DIN. Esses módulos empacotam as informações e cospem um sinal serial diferencial (**RS-485**), que é totalmente cego a ruídos elétricos.
O resultado? A informação de todos os 50 aparelhos varre a fábrica inteira através de **um único par trançado de fios (A e B)**. A empresa economiza milhares de reais em sensores caros e em quilômetros de cabeamento de cobre, mantendo a confiabilidade industrial.

---

3. Comutação de Potência e Controle Térmico

Função	Na Maquete (Didático)	Na Indústria (Planta Real)
Geração de Frio	Pastilhas Termoelétricas (Peltier)	Compressores Frigoríficos, Válvulas de Expansão
Geração de Calor	Resistor Cerâmico PTC (80W)	Resistências Tubulares Blindadas
Controle de Carga (PID)	Drivers de motor DC **BTS7960** modulando PWM.	**Controladores de Potência Tiristorizados (SCR)** com entrada 4-20mA ou **Inversores de Frequência**.
Relés Auxiliares (Sinal)	Módulos de relé 5V genéricos (optoacoplados).	**Relés de Interface Slim 24V** (ex: Finder Série 39, Phoenix Contact PLC-RSC).
Relé Principal (Força)	Módulo de relé 5V (usado no KM1).	**Contator Industrial 24V DC** (ex: Siemens Sirius) ou Relé de Estado Sólido (SSR) de alta potência.
Sensores de Temperatura	Sonda DS18B20 (1-Wire) e AM2315C (I2C).	Sondas **PT100/PT1000** ligadas a Transmissores de Temperatura (saída 4-20mA ou Modbus).

---

4. Infraestrutura, Energia e Painel (HMI)

Para garantir que o equipamento sobreviva ao ambiente fabril (poeira, umidade, vibração e ruído elétrico), a infraestrutura de montagem muda drasticamente.

Função	Na Maquete (Didático)	Na Indústria (Planta Real)
Fonte de Alimentação	Fonte chaveada genérica "Colmeia" (24V 10A).	**Fontes Industriais de Trilho DIN** (ex: Mean Well NDR/EDR, Phoenix Contact QUINT) com PFC ativo e proteção robusta.
Redutores de Tensão	Módulos Step-Down LM2596 (sem proteção).	**Conversores DC/DC Industriais DIN** isolados galvanicamente.
Gabinete (Painel)	Caixa de acrílico ou quadro de sobrepor plástico.	**Painéis Rittal / Steck IP55 ou IP65** (chapa de aço ou policarbonato) com ventilação forçada e filtros de ar.
Botões e Sinalização	Botões push-button 22mm genéricos e LEDs simples.	**Linha Industrial 22mm** (ex: Schneider Harmony XB4/XB5, WEG CEW) com blocos de contato independentes e vedação pesada.
Tela / IHM	Display genérico ES3C28P SPI/I2C	**IHM Industrial Touchscreen** (ex: WEG MT8000, Delta, Siemens KTP) comunicando

	acoplado ao ESP32.	via Ethernet ou RS-485 Modbus direto com o Arduino Portenta.
Distribuição Elétrica	Blocos de barramento impressos em 3D ou parafusos simples.	**Bornes Sak/Sindal de Passagem** de trilho DIN (ex: Phoenix Contact PT, WAGO) com molas *Push-in* antivibração.

Conclusão de Engenharia

A maquete foi desenhada para **provar o conceito lógico e arquitetural**. O código (Firmware), o protocolo de comunicação (Modbus/MQTT) e a topologia elétrica de segurança (Intertravamentos, Selo de Emergência) rodam exatamente da mesma maneira na Indústria. Apenas trocamos o "Tijolo Didático" pelo "Tijolo Industrial", mantendo a planta viva, altamente tecnológica e 100% aderente às normas.

🔙 **Anterior:** Doc 03 - Lista de Materiais (03_lista_materiais.md) | ➡ **Próximo:** Doc 10 - Base e Chão de Fábrica (../camada_1_maquete/10_base_e_chao_de_fabrica.md)